MegaSena em Clean Architecture: resumo

O projeto em Clean Architecture está pronto no `MegaSena-CleanArchitecture.zip`, com um README explicando a estrutura. Compilado com .NET 8, sem erros nem avisos. Os comandos foram executados e os números batem com os anteriores: 4.955.700 jogos válidos em 42 bolas e 2.651.467 em 38.

Estrutura (4 projetos, sem pacotes externos)

- **Domain:** regras puras. Tem `Jogo`, `Universo` (o globo), a regra `MaximoParesConsecutivos` e a contagem exata de jogos válidos.
- **Application:** casos de uso (gerar jogo, contar jogos, testar uniformidade, comparar universos) e as interfaces `IGeradorDeSorteio` e `IHistoricoRepository`.
- **Infrastructure:** o simulador físico 2D, o gerador aleatório rápido e a leitura do histórico em JSON.
- **Cli:** os comandos e o único lugar que escolhe as implementações concretas.

Como usar

```
dotnet run --project src/MegaSena.Cli -- sortear
dotnet run --project src/MegaSena.Cli -- contar
dotnet run --project src/MegaSena.Cli -- monte --n 5000000
dotnet run --project src/MegaSena.Cli -- uniformidade --n 300
```

Onde mexer

- O `historico.json` vem com números de exemplo. Troque pelos últimos sorteios reais e pelas 4 dezenas menos sorteadas.
- Os parâmetros do ventilador, da rotação e dos choques ficam no `simulador.json`.
- Para trocar o jeito de sortear, implemente `IGeradorDeSorteio`. Para ler o histórico de uma API ou de um banco, implemente `IHistoricoRepository`.

Limitação

Não há projeto de testes, porque o xUnit é um pacote NuGet e a geração foi feita sem

internet. O README sugere os primeiros testes a escrever.

Documentação relacionada

- `MegaSena.md` : regras, Monte Carlo, simulador de física e animação, com uma seção apontando para esta versão.

Próximos passos

1. Testes de unidade (xUnit) para `Jogo` , `ContadorDeJogosValidos` e `Universo` .
2. Trocar o histórico de exemplo por uma leitura dos resultados reais.